

002 -- Data Enrichment

Author: Wayne Kirk Schmidt
Email: wayne.kirk.schmidt@gmail.com

Purpose

`002_enrich.ipynb` is the second stage of the cryptocurrency statistical arbitrage research pipeline.

This stage constructs event-based datasets from the aligned price data produced in Stage 1, and transforms them into analysis-ready representations: synchronized lag vectors, daily returns, rolling volatility estimates, and standardized z-scores.

No filtering or strategy assumptions are applied here. All transformations remain reversible so that later notebooks can test multiple hypotheses against the same enriched dataset.

Inputs

From `output/001_download/`:

File	Description
<code>PRICES.pkl</code>	Long-format daily OHLCV for all assets
<code><COIN>.event_panel.pkl</code>	Per-asset panel of reference price and lagged target closes

Outputs

Written to `output/002_enrich/`:

File	Description
<code>events_raw.pkl</code>	All event panels concatenated, before synchronization
<code>events.pkl</code>	Synchronized event panel (consistent lag vector length)
<code>prices_raw.pkl</code>	Raw price data passed through for downstream reference
<code>price_wide.pkl</code>	Wide-format close price matrix (dates × coins)
<code>returns.pkl</code>	Daily percentage returns (pre-synchronization)
<code>returns_full.pkl</code>	Synchronized return matrix -- no missing values
<code>rolling_sigma.pkl</code>	Rolling 20-day volatility estimate per asset
<code>z_scores.pkl</code>	Standardized returns (return ÷ rolling sigma)

Pipeline Position

```
001_download -> data acquisition
002_enrich   <- you are here
-
003_analysis -> shock detection and cross-asset analysis
-
004_strategy -> signal construction
-
005_backtest -> execution and performance evaluation
```

Notes

- Sigma levels 0 and 1 are treated as statistical noise; events of interest begin at $\sigma \geq 2$.
- Filtering by shock magnitude is deferred to Stage 3 so that multiple thresholds can be evaluated independently.

1. Imports and Environment Setup

Standard library imports for data manipulation, file I/O, and path handling.

```
In [1]: import pandas as pd
import numpy as np
import pickle
from datetime import datetime, timedelta, UTC
import math
from pathlib import Path
```

2. Environment Configuration

Defines the asset universe, pipeline parameters, and output directories. Parameters are kept consistent across all stages.

Key parameters:

- `observation_window_length` -- number of lag days captured per event panel row
- `inspection_window` -- rolling window size (days) used for volatility estimation
- `holding_period` -- assumed trade holding period in days

```
In [2]: startdate = "2023-01-01"
trading_days = 252
frequency = "1d"
```

```

universe = [
    "BTCUSD", # Bitcoin
    "ETHUSD", # Ethereum
    "BNBUSD", # Binance Coin
    "SOLUSD", # Solana
    "XRPUSD", # Ripple
    "ADAUSD", # Cardano
    "DOGEUSD", # Dogecoin
    "AVAXUSD", # Avalanche
    "LTCUSD" # Litecoin
]

execution_delay = [0, 1, 2, 3]
execution_cost_bps = [20, 30, 40]

stage_label = "002_enrich"

OUTPUT_ROOT = Path("../output")
OUTPUT_ROOT.mkdir(parents=True, exist_ok=True)
MANIFEST_FILE = OUTPUT_ROOT / "manifest.pkl"

DOWNLOAD_DIR = OUTPUT_ROOT / "001_download"
DOWNLOAD_DIR.mkdir(parents=True, exist_ok=True)

ENRICH_DIR = OUTPUT_ROOT / "002_enrich"
ENRICH_DIR.mkdir(parents=True, exist_ok=True)

inspection_window = 20

observation_window_length = 10
observation_window = range(1, observation_window_length + 1)

holding_period = 1

```

3. Load or Initialize the Manifest

Loads the shared pipeline manifest. If this is the first stage to run, the manifest is initialized as an empty dictionary. The manifest tracks artifact paths and timestamps for every stage.

```

In [3]: MANIFEST_FILE = OUTPUT_ROOT / "manifest.pkl"

if MANIFEST_FILE.exists():
    manifest = pd.read_pickle(MANIFEST_FILE)
else:
    manifest = {}

manifest.setdefault(stage_label, {})

```

```

Out[3]: {}

```

4. Load and Combine Per-Asset Event Panels

Reads all `*.event_panel.pkl` files from Stage 1 and concatenates them into a single unified event dataset. Each row represents one (reference coin, date, target coin) tuple with a vector of forward closing prices.

```

In [4]: files = sorted(DOWNLOAD_DIR.glob("*.event_panel.pkl"))
print("files found:", len(files))
panels = []
for path in files:
    df = pd.read_pickle(path)
    panels.append(df)
events = pd.concat(panels, ignore_index=True)
print("rows:", len(events))
print("columns:", list(events.columns))
# persist raw event dataset
EVENTS_RAW_FILE = ENRICH_DIR / "events_raw.pkl"
events.to_pickle(EVENTS_RAW_FILE)
print("saved:", EVENTS_RAW_FILE)

```

```

files found: 9
rows: 74736
columns: ['ref_coin', 'date', 'ref_close', 'target_coin', 'target_close_lags']
saved: ../output\002_enrich\events_raw.pkl

```

5. Load the Master Price File

Loads the long-format `PRICES.pkl` from Stage 1. This gives us the full daily OHLCV series for all assets, which will be pivoted into a wide matrix for vectorized computation.

```

In [5]: PRICE_FILE = DOWNLOAD_DIR / "PRICES.pkl"

prices = pd.read_pickle(PRICE_FILE)

print("rows:", len(prices))
print("columns:", list(prices.columns))

coins = sorted(prices["coin"].unique())

print("number of coins:", len(coins))
print("coins:", coins)

prices.head()

# persist raw prices dataset
PRICES_RAW_FILE = ENRICH_DIR / "prices_raw.pkl"
prices.to_pickle(PRICES_RAW_FILE)

print("saved:", PRICES_RAW_FILE)

```

rows: 10984
columns: ['date', 'coin', 'open', 'close', 'volume']
number of coins: 9
coins: ['ADAUSD', 'AVAXUSD', 'BNBUSDT', 'BTCUSD', 'DOGEUSD', 'ETHUSD', 'LTCUSD', 'SOLUSD', 'XRPUSD']
saved: ..\output\002_enrich\prices_raw.pkl

5.1 Inspect the Combined Event Dataset

Quick shape and column check to confirm the concatenated event panel looks correct before proceeding.

```
In [6]: print(events.columns)
events.head()

Index(['ref_coin', 'date', 'ref_close', 'target_coin', 'target_close_lags'], dtype='str')

Out[6]:
```

	ref_coin	date	ref_close	target_coin	target_close_lags
0	ADAUSD	2023-07-14 00:00:00+00:00	0.3282	BTCUSD	[30288.46, 30225.15, 30154.58, 29864.32, 29925...
1	ADAUSD	2023-07-14 00:00:00+00:00	0.3282	ETHUSD	[1931.4, 1920.96, 1912.29, 1896.7, 1889.11, 18...
2	ADAUSD	2023-07-14 00:00:00+00:00	0.3282	BNBUSDT	[249.1, 241.9, 243.7, 239.1, 240.3, 241.5, 241...
3	ADAUSD	2023-07-14 00:00:00+00:00	0.3282	SOLUSD	[27.36, 27.39, 26.85, 25.57, 26.33, 25.33, 25....
4	ADAUSD	2023-07-14 00:00:00+00:00	0.3282	XRPUSD	[0.7157, 0.7488, 0.739, 0.7825, 0.8211, 0.7948...

6. Synchronize Lag Vectors

Enforces that every row in the event panel has a complete, full-length observation window. Rows from dates where any coin's lag vector is shorter than `observation_window_length` are dropped, ensuring all cross-asset comparisons are made over the same time horizon.

```
In [7]: # enforce synchronized lag vectors across all coins
# compute vector length
events["vector_length"] = events["target_close_lags"].apply(len)

print("vector length distribution:")
print(events["vector_length"].value_counts())

# find dates where every coin has a full observation window
valid_dates = (
    events.groupby("date")["vector_length"]
    .min()
)

valid_dates = valid_dates[valid_dates == observation_window_length].index

# keep only synchronized dates
events = events[events["date"].isin(valid_dates)].reset_index(drop=True)

# remove helper column
events = events.drop(columns=["vector_length"])

print("rows after synchronized vector validation:", len(events))
events.head()

# persist synchronized events dataset
EVENTS_FILE = ENRICH_DIR / "events.pkl"
events.to_pickle(EVENTS_FILE)
print("saved:", EVENTS_FILE)

# register artifact in manifest
manifest[stage_label]["events"] = str(EVENTS_FILE)
```

vector length distribution:
vector_length
10 74736
Name: count, dtype: int64
rows after synchronized vector validation: 74736
saved: ..\output\002_enrich\events.pkl

7. Pivot Prices to Wide Format

Converts the long-format price series into a wide matrix where rows are dates, columns are coins, and values are daily closing prices. This structure is required for vectorized cross-asset computation.

```
In [8]: price_wide = prices.pivot(index="date", columns="coin", values="close")
price_wide.head()

Out[8]:
```

	coin	ADAUSD	AVAXUSD	BNBUSDT	BTCUSD	DOGEUSD	ETHUSD	LTCUSD	SOLUSD	XRPUSD
date										
2023-01-01 00:00:00+00:00		0.24989	10.87	244.1924	16617.57	0.070069	1199.97	70.69	9.9994	NaN
2023-01-02 00:00:00+00:00		0.25336	11.15	245.2174	16677.87	0.071352	1214.05	74.80	11.2914	NaN
2023-01-03 00:00:00+00:00		0.25297	11.37	246.0136	16674.12	0.070514	1214.68	75.52	13.3944	NaN
2023-01-04 00:00:00+00:00		0.26796	12.07	259.2094	16849.97	0.073260	1256.89	75.46	13.4225	NaN
2023-01-05 00:00:00+00:00		0.26891	11.75	256.5466	16832.48	0.071411	1251.03	74.06	13.4253	NaN

8. Compute Daily Returns

Converts the synchronized price matrix into a return matrix using the standard percentage change formula:

$$r_t = (P_t / P_{t-1}) - 1$$

The resulting matrix captures the daily return for each asset on each date.

```
In [9]: returns = price_wide.pct_change()

RETURNS_FILE = ENRICH_DIR / "returns.pkl"
returns.to_pickle(RETURNS_FILE)
manifest[stage_label]["returns"] = str(RETURNS_FILE)
print("saved:", RETURNS_FILE)
print("shape:", returns.shape)
```

```
saved: ..\output\002_enrich\returns.pkl
shape: (1242, 9)
```

9. Save Wide Price Matrix

Persists the wide-format close price matrix to disk and registers it in the manifest. This artifact is used directly in later stages for price-level lookups.

```
In [10]: PRICE_WIDE_FILE = ENRICH_DIR / "price_wide.pkl"
price_wide.to_pickle(PRICE_WIDE_FILE)
manifest[stage_label]["price_wide"] = str(PRICE_WIDE_FILE)
print("saved:", PRICE_WIDE_FILE)
print("shape:", price_wide.shape)
```

```
saved: ..\output\002_enrich\price_wide.pkl
shape: (1242, 9)
```

10. Synchronize the Return Matrix

Drops all rows that contain any missing return values, ensuring the final matrix is fully populated across all assets. This is necessary because assets enter the dataset at different dates, and return calculations require at least one prior observation.

After this step, every remaining row contains valid returns for all 9 coins over a consistent time window.

```
In [11]: returns_full = returns.dropna(how="any").copy()
print("returns_full shape:", returns_full.shape)
print("first synchronized date:", returns_full.index.min())
print("last synchronized date:", returns_full.index.max())
returns_full.head()
RETURNS_FULL_FILE = ENRICH_DIR / "returns_full.pkl"
returns_full.to_pickle(RETURNS_FULL_FILE)
manifest[stage_label]["returns_full"] = str(RETURNS_FULL_FILE)
print("saved:", RETURNS_FULL_FILE)
```

```
returns_full shape: (1047, 9)
first synchronized date: 2023-07-15 00:00:00+00:00
last synchronized date: 2026-05-26 00:00:00+00:00
saved: ..\output\002_enrich\returns_full.pkl
```

11. Estimate Rolling Volatility

Computes a rolling standard deviation of returns over a 20-day window for each asset. This provides a time-varying estimate of each asset's recent volatility, which is used in the next step to normalize returns.

Rows with fewer than 20 prior observations are dropped, so the sigma matrix begins 20 trading days after the first synchronized return.

```
In [12]: rolling_sigma = returns_full.rolling(window=inspection_window).std()
rolling_sigma = rolling_sigma.dropna(how="any")
print("sigma matrix shape:", rolling_sigma.shape)
rolling_sigma.head()
SIGMA_FILE = ENRICH_DIR / "rolling_sigma.pkl"
rolling_sigma.to_pickle(SIGMA_FILE)
manifest[stage_label]["rolling_sigma"] = str(SIGMA_FILE)
print("saved:", SIGMA_FILE)
```

```
sigma matrix shape: (1028, 9)
saved: ..\output\002_enrich\rolling_sigma.pkl
```

12. Compute Standardized Returns (Z-Scores)

Normalizes each asset's daily return by its rolling volatility estimate:

$$z_t = r_t / \sigma_t$$

The resulting z-score measures how large a return is relative to the asset's recent historical variability. Large absolute z-scores ($|z| \geq 2$) indicate statistically unusual movements -- these are the shock events that drive the lead-lag strategy.

```
In [13]: aligned_returns = returns_full.loc[rolling_sigma.index]
z_scores = aligned_returns / rolling_sigma
print("z-score matrix shape:", z_scores.shape)
z_scores.head()
ZSCORE_FILE = ENRICH_DIR / "z_scores.pkl"
z_scores.to_pickle(ZSCORE_FILE)
manifest[stage_label]["z_scores"] = str(ZSCORE_FILE)
print("saved:", ZSCORE_FILE)
```

```
z-score matrix shape: (1028, 9)
saved: ..\output\002_enrich\z_scores.pkl
```

999. Persist the Manifest

Records all artifact paths and a completion timestamp for this stage into the shared manifest.

```
In [14]: manifest[stage_label]["timestamp"] = datetime.now(UTC).isoformat()
pd.to_pickle(manifest, MANIFEST_FILE)
print("manifest saved:", MANIFEST_FILE)
```

```
print("stages:", list(manifest.keys()))  
print("artifacts in stage:", list(manifest[stage_label].keys()))
```

manifest saved: ..\output\manifest.pkl

stages: ['001_download', '002_enrich']

artifacts in stage: ['events', 'returns', 'price_wide', 'returns_full', 'rolling_sigma', 'z_scores', 'timestamp']

In []: